Scrimmagine
Design Document

Jai Barath
Nick Van Wagner

Table of contents

Purpose 3
- Functional Requirements 4
- Non-Functional Requirements 6

Design Outline 8

Design Issues 10
- Functional Issues 10
- Non-Functional Issues 12

Design Details 14
- Descriptions of Classes and Interaction between Classes 15
- Sequence Diagrams 18

Purpose

Scrimmagine drops players into the biggest moments of real NFL games and lets them make the call. Every result is simulated from real historical plays, and the game scores you against the coach who actually made the decision.

Fans second-guess play calls every Sunday, but they never find out if their call would have worked. Full football sims make you play 60 minutes to reach the moments that matter. Our app skips straight to them: a turning-point finder scans a game's play-by-play data, ranks the plays by how much they swung win probability, and starts you right before each one.

After each call, an AI broadcast booth reacts with two lines of announcer commentary, praising a gutsy fourth-down conversion or roasting a timid punt. As a stretch goal, players can upload a screenshot of a broadcast and start a scenario from that exact moment.

Functional Requirements

* will be built after the core game works end to end if time allows

1. Scenario selection

As a player,

1. I would like each scenario to open with a brief intro that sets the scene.
2. I would like to start the game with one click to play with a set of random scenarios that updates daily.
3. I would like to have a separate option to play the game endlessly*.
4. I would like to upload a broadcast screenshot and start from that exact moment*.

2. Gameplay

As a player,

1. I would like to choose a play call: run, short pass, deep pass, punt, field goal, or go for it on fourth down.
2. I would like only the calls that make sense to be available (no punting on first down).
3. I would like each result to be drawn from real plays in similar situations, so outcomes feel realistic. I would like the initial set up for both the offense and defense to be realistic with previous data.
4. I would like to see the result of each play: yards gained, first down, turnover, or score.
5. I would like the current “round” to continue play by play until it ends in a score, turnover, punt, or the end of the half or game.
6. I would like to see my win probability update after every play.

3. Scoring against the real coach

As a player,

1. I would like to see what the real coach called and what actually happened after each of my decisions
2. I would like a running score comparing my win probability to the real coach's.
3. I would like an end-of-scenario summary with my final grade and best and worst calls.

4. AI broadcast booth

As a player,

1. I would like two lines of announcer commentary after every play reacting to my call and its result.
2. I would like the commentary to reference the real situation (score, clock, field position).
3. I would like the game to keep working if the AI commentary is slow or fails.

5. Screenshot mode (stretch)

As a player,

1. I would like the app to read the score bug from my screenshot: teams, score, quarter, clock, and down and distance.
2. I would like the app to match those details to the exact play in the database to get field position and context.
3. I would like to confirm or correct the detected situation before the scenario starts.
4. I would like the commentary to mention what the defense appears to show pre-snap. (stretch)

Non-Functional Requirements

1. Performance

As a developer,

1. I would like play simulation to respond within 300 ms, since it only samples from data already in memory.
2. I would like AI commentary to arrive within 3 seconds, loading separately so it never blocks the next play call.
3. I would like the scenario list to load within 1 second, served from precomputed JSON.
4. I would like screenshot parsing to finish within 10 seconds. (stretch)

2. Reliability

As a developer,

1. I would like the game to stay playable if the LLM API fails, falling back to simple template commentary.
2. I would like all play-by-play data and scenarios loaded at startup, so the app never depends on an outside data source during the demo.
3. I would like the simulator to always return a valid result, even for rare situations with few matching plays.

3. Usability

As a developer,

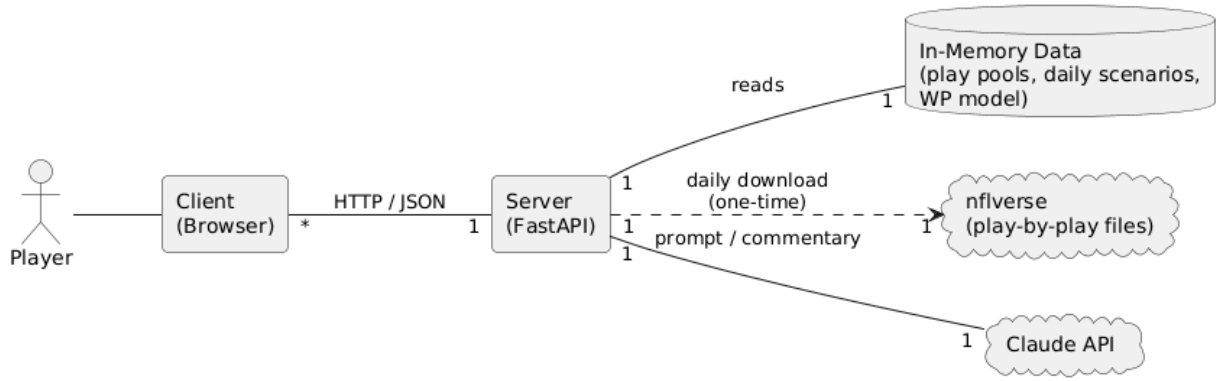
1. I would like a first-time player to make their first call within 15 seconds of opening the app.
2. I would like the game to work on a laptop browser and be usable on a phone.
3. I would like every play to show clear feedback: what happened, the win probability change, and the real coach's call.

4. Security

As a developer,

1. I would like the LLM API key stored only on the server in an environment variable, never sent to the browser.
2. I would like uploaded screenshots checked for file type and size (under 5 MB), processed in memory, and not stored.
3. I would like basic rate limiting on AI endpoints so a public demo link can't run up API costs.

High level overview of the system



Our implementation will use a client-server architecture with 3 layers: the client-facing browser, the FastAPI server for game logic, and in-memory data layer loaded at startup. Unlike a traditional 3-tier architecture, we do will not use a separate database since the data is read-only and small enough to fit in memory

nflverse contains various datasets relating to NFL, providing the info needed for our game logic and, most importantly, to compute our own win probability (WP) model. While each scenario starts from a real moment in an NFL game, each decision the player makes creates a new scenario with its own down, distance, field position, and win probability, which the server calculates from similar real plays.

Only the server calls the Claude API to keep our key private. To limit calls, each day's scenario intros will be generated once.

Design Issues

Functional Issues

1. Finding turning points: We rank plays by win probability added (WPA) and leverage in code, then have an LLM write the intros.nflverse data already includes WPA for every play, so ranking takes a few lines of pandas and gives the same answer every time. An LLM reading hundreds of plays would be slower, cost more, and could miss or invent moments. We still use the LLM where it's strongest: writing the scene-setting intro for each daily scenario.

2. Simulating a play call: We sample a real historical play from similar situations with the same play type. Sampling real plays keeps outcomes realistic by definition, including the right mix of sacks, incompletions, and big gains. It's simple to build and easy to explain. A physics engine wouldn't fit in 8 hours, and LLM-generated results would be inconsistent and hard to defend.

3. Scoring against the real coach: We compare the player's total WPA across the scenario with the real coach's. Final outcomes alone are noisy, since a good call can still fail. WPA rewards good decisions on every play and is the standard measure in football analytics, so it makes for a fairer score.

4. Screenshot mode (stretch): A vision model reads only the score bug, then we look up the exact play in our play-by-play data. Vision models read the score bug reliably but often misjudge field position from the yard-line numbers. Teams, quarter, clock, score, and down and distance together nearly always identify a single play, which gives us exact field position and what really happened next.

Non-Functional Issues

1. Backend framework: Python with FastAPI. The data work (nflverse, pandas, scikit-learn) is all in Python, so keeping the backend in Python avoids a second language and data handoffs. FastAPI is quick to set up and documents its own endpoints.

2. Win probability for new situations: We train our own win probability model on nflverse data, with a lookup-table fallback. Once a player makes a different call than the real coach, the game reaches states that aren't in the data. A gradient boosting model trained on score difference, time remaining, field position, down, distance, and timeouts covers any state and trains in about a minute. If it misbehaves, we fall back to average real win probabilities grouped by score, time, and field position.

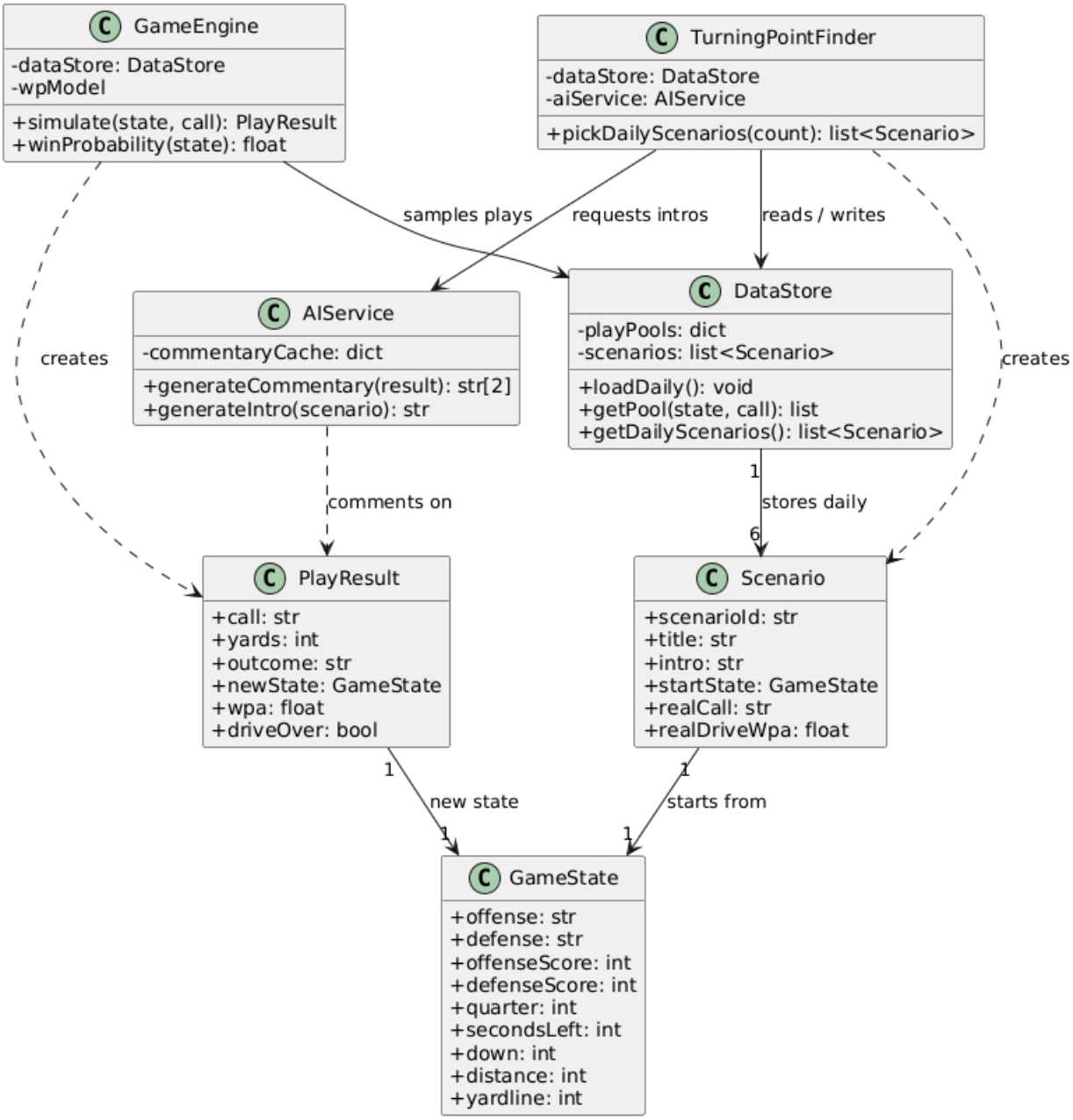
3. Where data lives: In memory, loaded from files downloaded once per day. The data is read-only and small enough to fit in memory once unused columns are dropped.

Skipping a database saves setup time, keeps every play fast, and removes a point of failure during the demo.

4. AI model: The Claude API. We need fast, reliable text generation plus image input for screenshot mode, with structured JSON output. A hosted API is the only practical way to get that on hackathon laptops, and generating intros once per day while caching commentary keeps the cost low.

Design Details

Descriptions of Classes and Interaction between Classes



- DataStore
    - Holds the day's data in memory: the play pools grouped by situation and the 6 daily Scenarios.
    - Is loaded once per day through loadDaily(), which reads the downloaded nflverse data and builds the play pools.

    - Gives GameEngine the matching pool of real plays for a game state and play call through getPool().
    - Returns the 6 daily Scenarios to the client through getDailyScenarios().
- GameState
    - Represents the situation before a play: offense, defense, both scores, quarter, seconds left, down, distance, and yard line.
    - Is the starting point of every Scenario and is returned as the new state inside every PlayResult.
    - Is passed into GameEngine to simulate a play and to compute win probability.
- Scenario
    - Represents one playable moment from a real NFL game.
    - Contains a unique scenario ID, a title, an AI-written intro, the starting GameState, the real coach's call, and the real coach's total WPA on that drive.
    - Is created by TurningPointFinder and stored in DataStore, which holds 6 per day.
    - Uses realCall and realDriveWpa to compare the player's decisions with the real coach's.
- PlayResult
    - Represents the outcome of one play call: the call, yards gained, outcome type, the new GameState, the WPA from the play, and whether the drive is over.
    - Is created by GameEngine after every play and returned to the client.
    - Is passed to AIService to generate commentary.
- GameEngine
    - Runs the game logic for each play and holds the trained win probability model.
    - simulate() takes a GameState and a play call, gets the matching pool from DataStore, samples a real play, applies it, and returns a PlayResult.
    - winProbability() returns the offense's chance of winning from a GameState, used before and after each play to compute WPA.
- TurningPointFinder
    - Picks the 6 daily Scenarios through pickDailyScenarios().
    - Reads plays from DataStore, filters out garbage time, ranks plays by WPA and leverage, and keeps the top play per drive.
    - Asks AIService to write an intro for each Scenario, then stores the Scenarios in DataStore.
- AIService
    - The only class that calls the Claude API.
    - generateIntro() writes a short scene-setting intro for each daily Scenario, once per day.
    - generateCommentary() takes a PlayResult and returns two announcer lines.
    - Caches commentary for repeated situations and uses template lines if the API is slow or fails.

Data source

nflverse play-by-play for the 2024 and 2025 seasons, loaded with `nflreadpy` (`load_pbp([2024, 2025])`) or downloaded directly as Parquet from the nflverse-data GitHub releases. Download once, keep only the columns below, and save to `data/pbp.parquet`.

Columns used: `game_id`, `play_id`, `season`, `week`, `home_team`, `away_team`, `posteam`, `defteam`, `qtr`, `game_seconds_remaining`, `half_seconds_remaining`, `down`, `ydstogo`, `yardline_100`, `score_differential`, `posteam_timeouts_remaining`, `defteam_timeouts_remaining`, `play_type`, `pass_length`, `yards_gained`, `touchdown`, `interception`, `fumble_lost`, `sack`, `field_goal_result`, `kick_distance`, `wp`, `wpa`, `desc`, `drive`, `total_home_score`, `total_away_score`, `result`.

Data models (Pydantic)

Model	Fields	Purpose
`GameState`	offense, defense, offense_score, defense_score, quarter, seconds_left_in_game, down, distance, yardline_100, offense_timeouts, defense_timeouts	The situation before a play. `yardline_100` = yards from the opponent's end zone, matching nflverse.
`PlayCall`	enum: `run`, `short_pass`, `deep_pass`, `punt`, `field_goal`	The player's decision. Going for it on 4th = calling a run or pass on 4th down.
`PlayResult`	call, yards, outcome (`gain`, `first_down`, `touchdown`, `turnover`, `turnover_on_downs`, `punt`, `fg_made`, `fg_missed`), description, sampled_play_id	What happened, plus the real play it was sampled from.
`SimulateResponse`	result, new_state, wp_before, wp_after, wpa, drive_over	Returned after every call.
`Scenario`	id, game_id, title, intro, start_state, real_call, real_play_desc, real_drive_wpa, leverage_score	One playable starting moment. `real_drive_wpa` = the real coach's total WPA from this play to the end of the drive.
`Commentary`	line_1, line_2	Two announcer lines.

Turning point finder

Runs offline to build `data/scenarios.json`, and live for the game picker.

1. Load all plays for one `game_id`. Keep real plays only: `play_type` in run, pass, punt, field_goal.
2. Drop garbage time: skip plays where pre-play `wp` is below 0.05 or above 0.95.
3. Score each play: `leverage_score = abs(wpa)`, times 1.5 if it's 4th down, times 1.25 if under 5 minutes are left in the 2nd or 4th quarter.
4. Keep only the highest-scoring play per drive, so you don't get three scenarios from the same drive.
5. Return the top N (default 5) sorted by score, in game order for display.
6. For each one, compute `real_drive_wpa` as the sum of `wpa` from that play to the end of its drive.
7. Send the state and real outcome to the LLM to write a 2 to 3 sentence intro that sets the scene without spoiling the result.

Play simulator

Given a `GameState` and a `PlayCall`, sample a real outcome.

Situation buckets:

- Down: 1, 2, 3, 4
- Distance: 1, 2 to 3, 4 to 6, 7 to 10, 11+
- Field zone by `yardline_100`: 80 to 99 (backed up), 50 to 79 (own side), 21 to 49 (opponent side), 1 to 20 (red zone)

By call:

- `run`: sample from `play_type == run` in the same bucket.
- `short_pass` and `deep_pass`: sample from `play_type == pass` with matching `pass_length`. Sacks have no `pass_length`, so add sacks from the bucket to both pools at their real rate.
- `punt`: sample net yards from real punts in the same field zone.
- `field_goal`: kick distance = `yardline_100 + 17`. Make chance = real make rate for kicks within 3 yards of that distance.

Fallbacks: if a bucket has fewer than 30 plays, drop the field zone, then merge distance buckets, until there are at least 30.

Applying a result: cap yards at the end zone (touchdown) and at a safety. Update down and distance, flip possession on turnovers, punts, and turnover on downs. Clock: subtract 30 seconds for a run or completion, 6 for an incompletion, and 15 for any play in the last 2 minutes of a half. This is a simplification we'll mention to judges.

Drive ends on a touchdown, field goal attempt, turnover, punt, turnover on downs, or the clock running out in the half.

Win probability model

- **Features:** score differential, game seconds remaining, score differential divided by (seconds remaining + 1), yardline_100, down, distance, offense timeouts, defense timeouts.
- **Label:** whether the team with the ball won (from `result` and the home/away teams). Ties count as 0.5 weight on each class, or drop them.
- **Model:** scikit-learn `HistGradientBoostingClassifier` (handles the nonlinear late-game effects better than logistic regression and trains in about a minute). Save with joblib.
- **Sanity check:** compare our WP with nflverse's `wp` column on held-out games. Within about 0.05 on average is good enough.
- **Fallback:** average nflverse `wp` by score differential, time bucket, and field zone.
- For a state where the defense now has the ball, WP for the player's team = 1 minus the model's output for the new offense.

API endpoints

Method	Path	Body or params	Returns
GET	`/api/scenarios`	none	Curated `Scenario` list from `scenarios.json`
GET	`/api/games`	`season`, `team` (optional)	Games with teams, week, final score
GET	`/api/games/{game_id}/turning-points`	`limit` (default 5)	`Scenario` list for that game
POST	`/api/simulate`	`state`, `call`	`SimulateResponse`
POST	`/api/commentary`	`state_before`, `call`, `result`, `wp_before`, `wp_after`	`Commentary`
POST	`/api/scenario-from-screenshot` (stretch)	image file	Detected fields plus the matched `Scenario`

AI prompts

- **Commentary:** system prompt casts the model as a two-person TV booth (play-by-play plus color analyst). Input is the situation, the call, the result, and the WP swing. Output is JSON with `line_1` and `line_2`, each under 25 words. Tone: hype good aggressive calls, roast conservative calls that backfire, stay PG. Use a fast model with a 3-second timeout and template fallbacks like "And the drive stalls there."

- **Scenario intro:** 2 to 3 sentences, broadcast voice, ends right before the snap, never reveals what happened.
- **Screenshot (stretch):** vision prompt returns JSON with `away_team`, `home_team`, `away_score`, `home_score`, `quarter`, `clock`, `down`, `distance`, and `offense` (if it can tell). The backend matches on teams, quarter, clock within 5 seconds, down, distance, and score to find the `play_id`.

Sequence of events

Player starts a curated scenario

1. The client requests `GET /api/scenarios`, and the server returns the list from memory.
2. The player picks one, and the client shows the field, score bug, and AI intro.
3. The client enables only the legal calls for the current down and field position.

Player makes a call

1. The client sends `POST /api/simulate` with the state and the call.
2. The server buckets the situation, samples a real play, applies the result, and runs the WP model on both states.
3. The server returns the result, new state, and WPA. The client animates the play and updates the WP meter and score.
4. At the same time, the client sends `POST /api/commentary` and shows the lines when they arrive.
5. If `drive_over` is true, the client shows the summary: your WPA vs. `real_drive_wpa`, the real coach's call, and what actually happened.

Player picks a game's turning points

1. The client requests `GET /api/games`, and the player picks a game.
2. The server runs the turning point finder for that `game_id` and asks the LLM for an intro per scenario.
3. The client shows the ranked scenarios, and play continues as above.

Player uploads a screenshot (stretch)

1. The client sends the image to `POST /api/scenario-from-screenshot`.
2. The server sends it to the vision model and gets back the score bug fields as JSON.
3. The server searches the play-by-play data for the matching play and builds a `Scenario` from it.
4. The client shows the detected situation for the player to confirm, then starts the scenario.

Notes for Claude Code

Build the backend only. The UI is designed separately in Claude Design and calls the endpoints above, so keep the API contract stable and return JSON shaped exactly like the models.

Repo structure

```
None
backend/
  app/
    main.py            # FastAPI app, CORS, routes, startup loading
```

```
    models.py          # Pydantic models
    data_loader.py     # load pbp.parquet + scenarios.json into
memory
    turning_points.py  # ranking + per-drive dedupe
    simulator.py       # buckets, sampling, applying results, clock
    wp_model.py        # load model, predict WP for a GameState
    ai.py              # commentary, intros, screenshot parsing +
fallbacks
  scripts/
    download_data.py   # fetch 2024-2025 pbp, trim columns, save
parquet
    train_wp.py        # train + evaluate + save wp_model.joblib
    build_scenarios.py # run finder on demo games, write
scenarios.json
  data/                # pbp.parquet, wp_model.joblib,
scenarios.json
  tests/               # simulator + turning point tests
  requirements.txt
  .env.example         # ANTHROPIC_API_KEY
```

Build order

1. `download_data.py`, then confirm the columns exist and print a few rows.
2. `models.py` and `simulator.py`, with tests: every call returns a valid result, buckets fall back correctly, touchdowns and turnovers flip state correctly.
3. `train_wp.py` and `wp_model.py`, with the sanity check against nflverse `wp`.
4. `POST /api/simulate` working end to end, tested with curl.
5. `turning_points.py`, `build_scenarios.py`, and `GET /api/scenarios`.
6. `ai.py` and `POST /api/commentary`, with a timeout and template fallback.
7. `GET /api/games` and `GET /api/games/{game_id}/turning-points`.
8. Stretch: `POST /api/scenario-from-screenshot`.

Rules

- Load all data once at startup, never per request.
- Never expose the API key to the client.
- Every AI call needs a timeout and a non-AI fallback, so the game still works offline.
- Keep functions small and typed. We'll be explaining this code to judges.

UI Design:

We want the theme to be retro, so take inspiration from games like krillion (https://krillion.io/) or Eraly (https://eraly.io/)

There is a zip file in the git called "scrimmagine-mockups" with the HTML mockups for the app.

Each team should have their team colors on their players, the UI will have a top down view of the field, and an animation of what happened in the prediction will play out on the screen. (Think something like retro bowl)



The choices will be at the bottom to click from, and the WP and increase will also be shown after every play.

Use sprites in the spite folder, and make more when necessary.
If the screenshot upload is implemented, then have a button that creates a pop up where you can insert an image, and then the AI model will create the scenario back on the main screen.

Sound Design:
Sound design should also include noises for button presses that sound retro. Including noises for good results and bad results. Could include noises like low-quality cheering and booing, grunts, and overall retro sound design.

Auditing/Testing (disregard for now)

use open ai to audit public repo or zip file for private repo

Deployment (disregard for now)

“Deploy, make no mistakes”

Give us options for how we can deploy our game.

Docker
Vercel